Zargux — Incident Detection in Linux Servers

USE-CASE DOCUMENTATION

A report submitted in partial fulfillment of the requirements

for the degree of Bachelor of Technology in

Computer Science & Engineering / Cyber Security

Submitted by

[Student Name] — [Roll / Enrollment No.]

[Team member 2 — Roll No.] (delete if solo)

Under the guidance of

[Internal Faculty Guide — Name & Designation]

Department of Computer Science & Engineering

ABES Engineering College, Ghaziabad

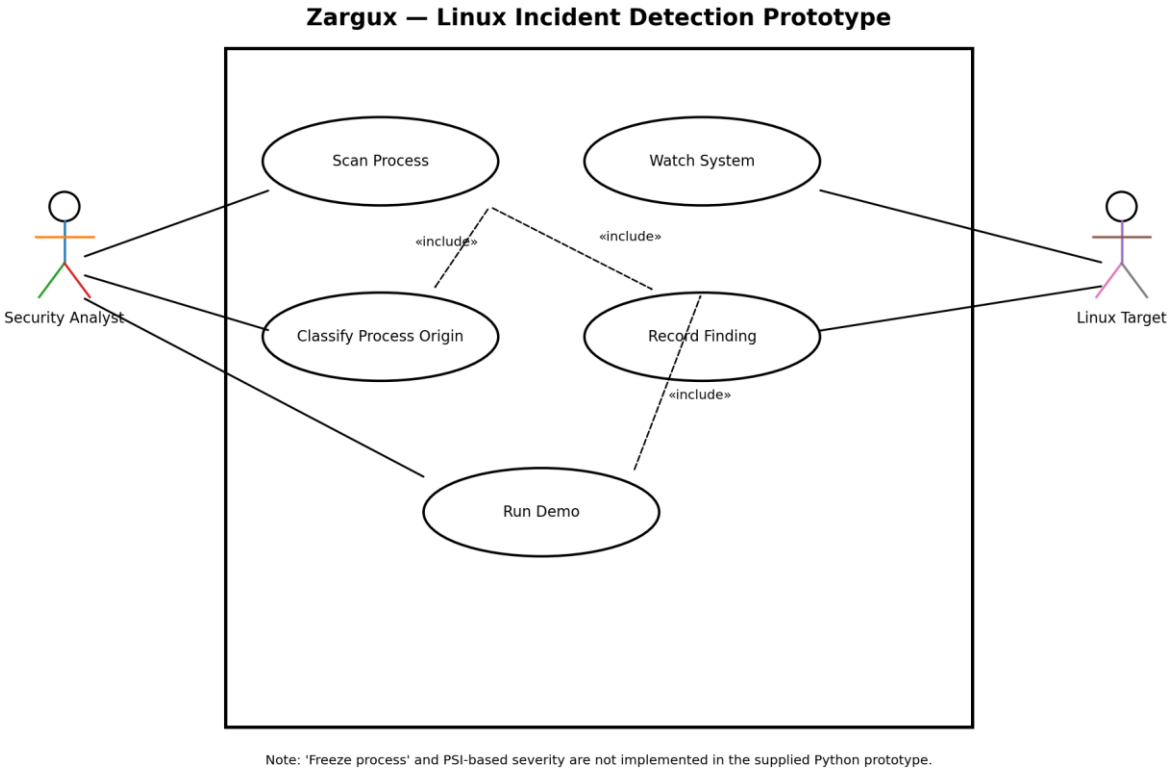
Dr. A.P.J. Abdul Kalam Technical University, Lucknow

Session 2025–26

1. Use-Case Diagram

» Draw in draw.io / Visual Paradigm and paste here. Conventions:

- Actors = stick figures OUTSIDE the system box (e.g. Analyst, Attacker, the Target System/Timer).
- System boundary = a labelled rectangle around all use cases.
- Use cases = ovals named with a verb phrase (Run Detection, Generate Alert, Respond to Incident).
- «include» = dashed arrow to an always-used sub-case; «extend» = optional behaviour; association = solid line.



2. Use-Case Descriptions

» One table per use case. A filled example is below; copy the blank table for each of your use cases.

2.1 Example (filled) — Run Detection

Use Case ID	UC-01

Use Case Name	Run Detection
Actor(s)	Security Analyst (primary); Target System (secondary)
Description	The analyst starts the detector, which watches evidence and raises an alert when the rule fires.
Preconditions	Detector configured; evidence source (logs/events) available.
Trigger	Analyst runs the detector.
Main Flow	1. Analyst starts detector. 2. System reads evidence. 3. System evaluates the rule. 4. On match, system raises an alert with the incident details.
Alternate Flow	3a. No match within window → system keeps watching.
Exception Flow	2a. Evidence source unavailable → system logs an error and retries.
Postconditions	An incident record exists; analyst is alerted.
Priority	High

2.2 Blank table — copy one per use case

Use Case ID	[UC-0x]
Use Case Name	[verb phrase]
Actor(s)	[primary; secondary]
Description	[one line]
Preconditions	[…]
Trigger	[…]
Main Flow	1. … 2. … 3. …
Alternate Flow	[…]

Exception Flow	[…]
Postconditions	[…]
Priority	[High/Medium/Low]

» Suggested use cases for a detection project: Run Detection, Simulate Attack, Generate Alert, Respond to Incident, View Dashboard.

Zargux — Use-Case Documentation (Python Prototype)

Project context. Zargux is an incident-detection prototype for Linux servers. The supplied project definition describes origin-based detection of programs executing from writable temporary locations such as /dev/shm, /tmp, and /var/tmp, with CPU pressure used as contextual evidence. The supplied Python prototype implements process-origin classification, live monitoring, persistence to findings.json, and a three-case demonstration.

1. Scope of the Current Prototype

- Runs inside an Ubuntu 24.04 LTS Multipass VM.
- Reads /proc/<pid>/exe to determine the executable origin of running processes.
- Treats /dev/shm/, /tmp/, /var/tmp/, and executables ending in ' (deleted)' as HIGH.
- Uses dpkg -S to identify package-managed executables and classifies them as CLEAR.
- Classifies other non-package-managed executables outside writable temporary locations as WATCH.
- Reads /proc/pressure/cpu and displays avg10, avg60, and avg300 in watch mode.
- Persists findings in findings.json, keeping the most recent 50 records.
- Provides scan, watch, and demo CLI commands.

2. Use-Case Descriptions

Use Case ID	UC-01
Use Case Name	Scan Process
Actor(s)	Security Analyst (primary); Linux Target (secondary)
Description	Perform a one-shot origin-based security verdict for a selected process ID.
Preconditions	Zargux is installed on the Ubuntu VM and the PID exists and is readable.
Trigger	Analyst runs sudo python3 zargux.py scan

	<pid>.
Main Flow	1. Read /proc/<pid>/exe. 2. Classify the executable origin. 3. Print PID, executable path, verdict, and reason. 4. Save the finding to findings.json.
Alternate Flow	The process may exit between PID discovery and /proc lookup; Zargux reports that the PID was not found or permission was denied.
Exception Flow	Permission is insufficient or the process exits; the scan terminates with an error.
Postconditions	A verdict is displayed and a finding is persisted.
Priority	High

Use Case ID	UC-02
Use Case Name	Watch System
Actor(s)	Security Analyst (primary); Linux Target (secondary)
Description	Continuously monitor CPU pressure and process origins and display processes whose verdict is WATCH or HIGH.
Preconditions	Zargux has permission to inspect the target processes; /proc is available.
Trigger	Analyst runs sudo python3 zargux.py watch [--interval N].
Main Flow	1. Read CPU PSI values. 2. Enumerate numeric PIDs under /proc. 3. Read each /proc/<pid>/exe link. 4. Classify each process. 5. Hide CLEAR processes. 6.

	Display WATCH/HIGH processes and CPU percentage. 7. Save a new finding when a PID's verdict changes. 8. Repeat after the configured interval.
Alternate Flow	No suspicious processes are present; the display reports that nothing is flagged.
Exception Flow	PSI cannot be read; Zargux displays that CPU pressure is unavailable but continues the process scan.
Postconditions	The terminal provides a live view and findings are persisted.
Priority	High

Use Case ID	UC-03
Use Case Name	Classify Process Origin
Actor(s)	Zargux Detection Engine (primary); Linux /proc; dpkg
Description	Determine whether a running executable is deleted, located in a writable temporary path, package-managed, or otherwise unpackaged.
Preconditions	An executable path has been obtained from /proc/<pid>/exe.
Trigger	The scan/watch logic calls classify(exe_path).
Main Flow	1. Check whether the path ends with ' (deleted)'. 2. Check whether it begins with /dev/shm/, /tmp/, or /var/tmp/. 3. If not, query dpkg -S for package ownership. 4. Return HIGH, CLEAR, or WATCH with

	a reason.
Alternate Flow	A package lookup may return no match; the process becomes WATCH unless its path is suspicious.
Exception Flow	The executable path cannot be read; the caller skips or reports the process depending on the command.
Postconditions	A normalized verdict and human-readable reason are returned.
Priority	High

Use Case ID	UC-04
Use Case Name	Record Finding
Actor(s)	Zargux (primary); findings.json
Description	Persist security findings so that scan, watch, and demo results can be reviewed after detection.
Preconditions	The working directory is writable.
Trigger	A scan, watch transition, or demo case produces a finding.
Main Flow	1. Load existing findings.json if present. 2. Append time, summary, and severity. 3. Keep the most recent 50 records. 4. Write the updated JSON file.
Alternate Flow	If findings.json contains invalid JSON, Zargux starts with an empty list and continues.
Exception Flow	The file cannot be written; persistence fails and the operating system reports the error.

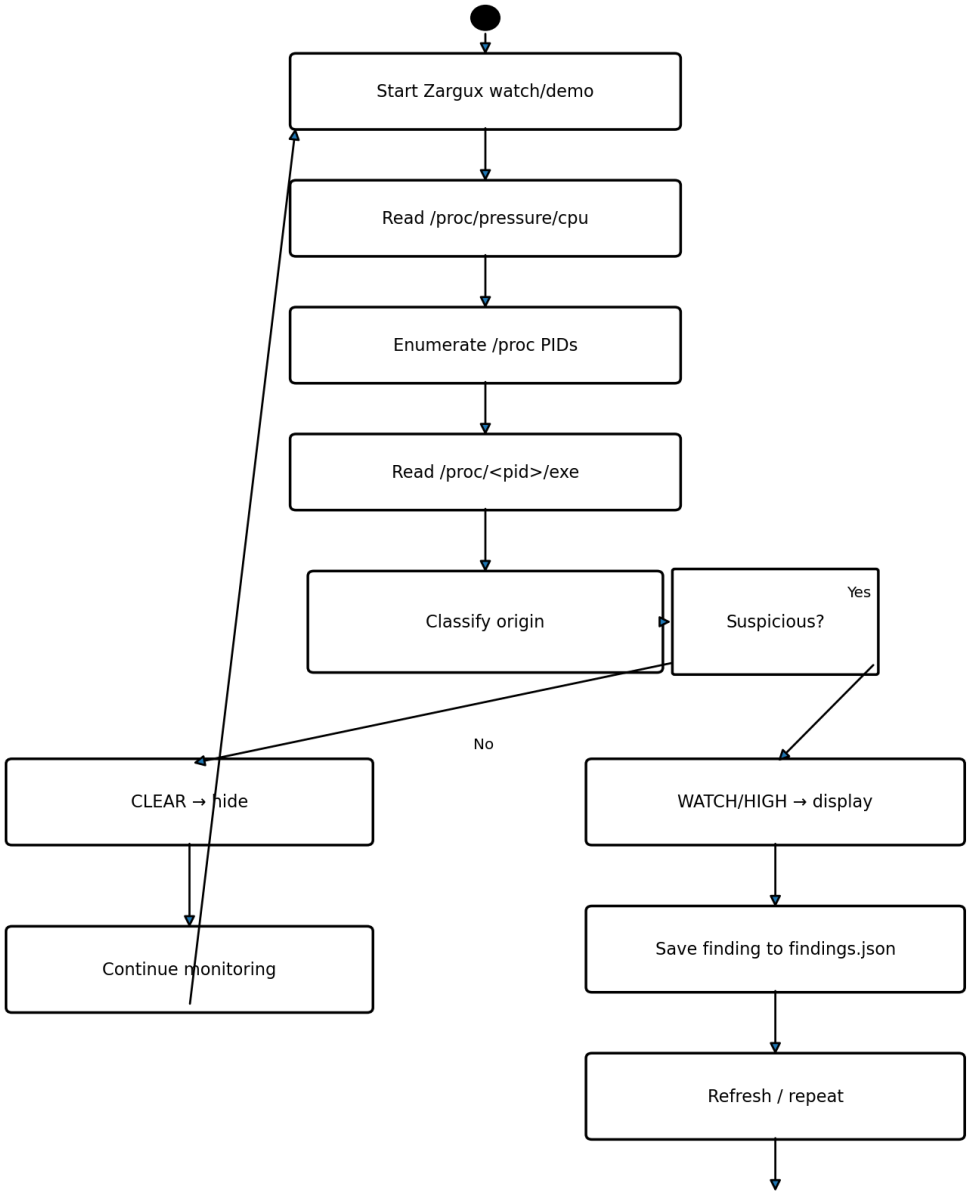
Postconditions	The finding is stored with a timestamp, summary, and severity label.
Priority	Medium

Use Case ID	UC-05
Use Case Name	Run Three-Case Demo
Actor(s)	Security Analyst (primary); Linux Target (secondary)
Description	Run the presentation workflow that contrasts a legitimate stress-ng process, a copied stress-ng binary in /dev/shm, and a legitimate unpackaged control.
Preconditions	Ubuntu VM has stress-ng, systemd, sudo access, and the required paths available.
Trigger	Analyst runs sudo python3 zargux.py demo.
Main Flow	1. Stop leftovers from previous demo runs. 2. Launch stress-ng from /usr/bin. 3. Copy stress-ng to /dev/shm and launch the copy. 4. Create ~/bin/my-legit-tool from /bin/sleep and launch it. 5. Classify all three processes and save findings. 6. Print a side-by-side results table. 7. Stop demo services and remove the /dev/shm copy.
Alternate Flow	A demo process may fail to start or disappear before classification; that case is omitted from the results.
Exception Flow	Required sudo/systemd/stress-ng operation fails; the demo stops with an error.

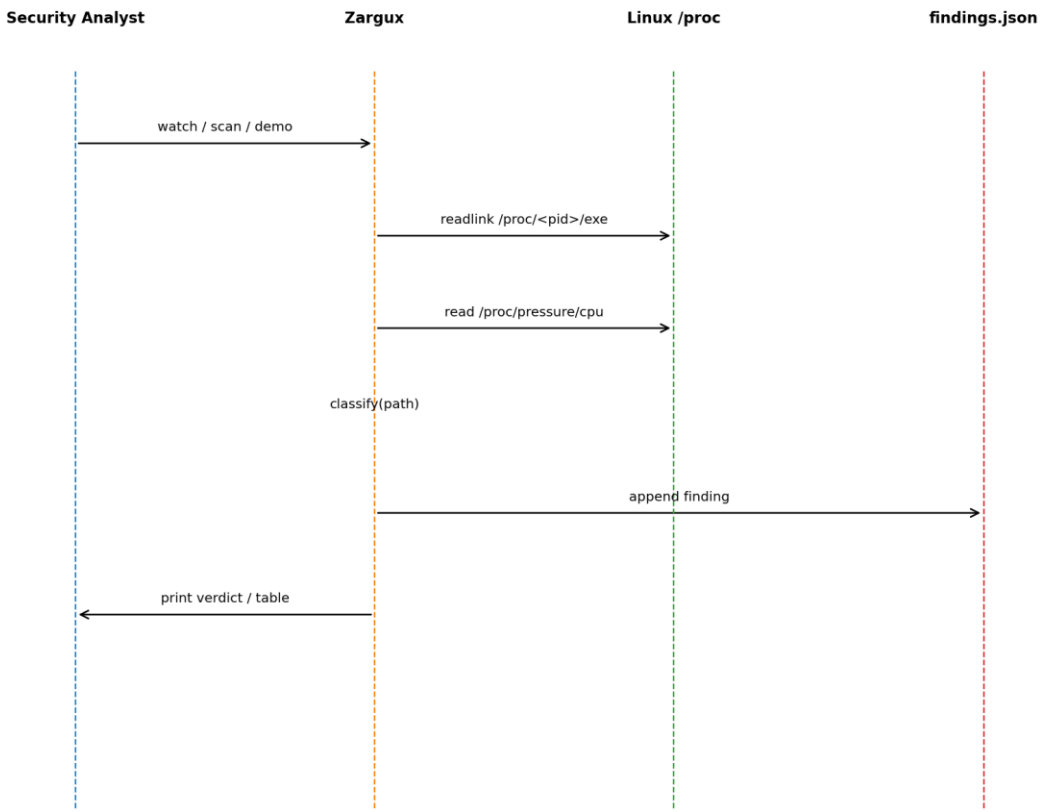
Postconditions	The terminal shows how executable origin changes the verdict even for the same stress-ng binary and similar CPU workload.
Priority	High

3. Activity Diagram

Current prototype flow for live monitoring.

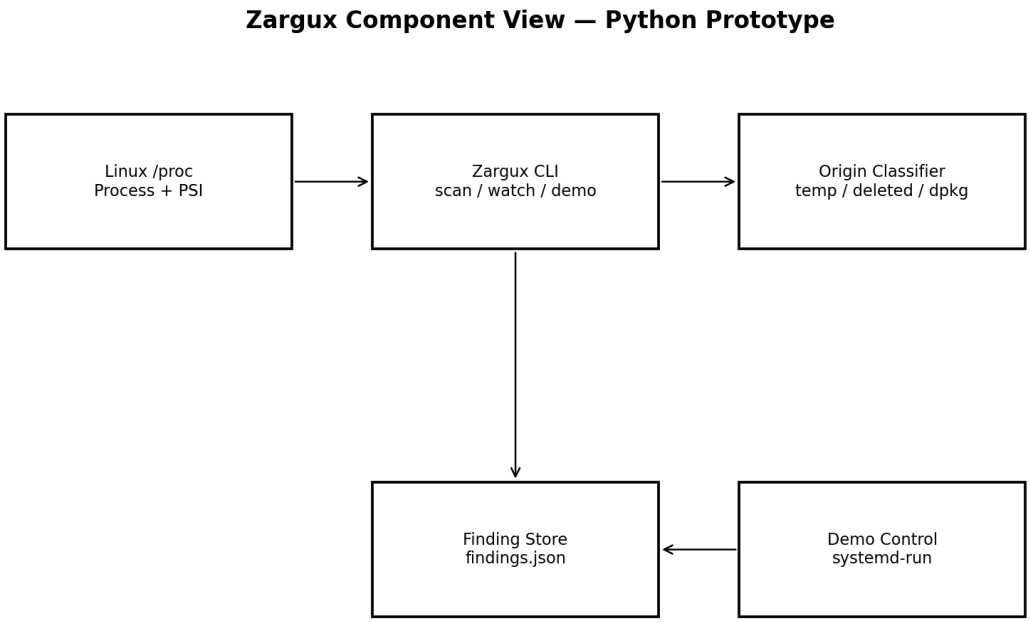


4. Sequence Diagram

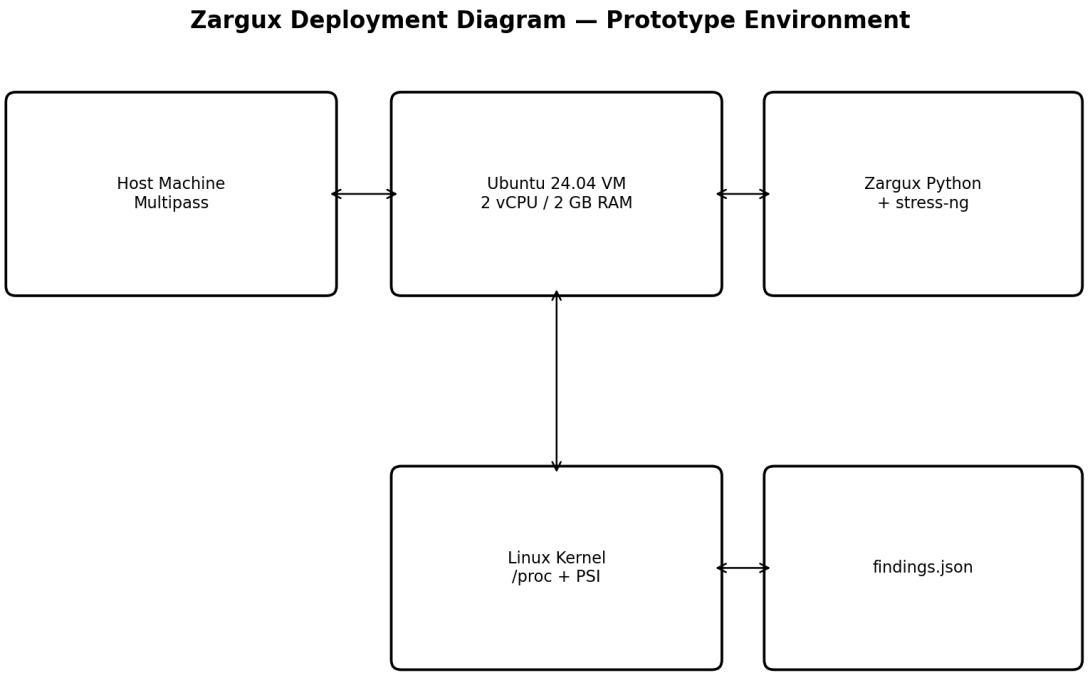


Current Python prototype: origin classification drives verdict; PSI is read/displayed but not used by classify().

5. Component Diagram



6. Deployment Diagram



7. Implementation Notes and Prototype Gaps

The project-definition document and the supplied Python code are not fully identical. For academic accuracy, the current prototype should be described separately from planned/future Zargux behavior.

CPU PSI severity: Project definition: HIGH when CPU PSI avg10 is at/above 40 for 10 seconds. Supplied code: read_psi() displays PSI, but classify() does not use PSI to calculate the verdict or severity.

Response action: Project definition says the tool saves evidence and freezes the suspicious process. Supplied code saves a finding, but it does not freeze/stop the detected process. The demo stops its own systemd services only during cleanup.

Monitoring interval: Project definition states every 2 seconds. The supplied watch command defaults to a 3.0-second interval unless --interval is supplied.

Evidence file: The supplied code persists findings in findings.json containing time, summary, and severity. It does not currently create a separate executable/evidence copy of the suspicious process.

Language/runtime: The project definition describes the intended detector as Zig 0.13.0, while the supplied implementation is a Python 3 prototype executed inside the Ubuntu VM.

8. Suggested Final Wording for the Current Prototype

Zargux is a Linux incident-detection prototype that identifies suspicious running processes by inspecting the origin of their executable through /proc/<pid>/exe. Executables running from writable temporary locations such as /dev/shm, /tmp, and /var/tmp, or executables that have been deleted from disk while still running, are classified as HIGH. Package-managed executables are classified as CLEAR, while other unpackaged executables outside those locations are classified as WATCH. Zargux can scan a single PID, continuously watch the system, persist findings to findings.json, and run a controlled three-case demonstration in an Ubuntu 24.04 Multipass VM. The

prototype also reads CPU pressure from /proc/pressure/cpu as contextual monitoring information.

9. Reference Environment

- Host VM manager: Multipass 1.16.4.
- Target: Ubuntu 24.04 LTS, 2 vCPU, 2 GB RAM.
- Attack/demo utility: stress-ng 0.20.01-1.
- Package ownership check: dpkg.
- Prototype implementation: Python 3 CLI.
- Intended implementation in the project definition: Zig 0.13.0.